

CustomGate 360

Documentación del Core — Backend Spring Boot

1. Arquitectura General

El proyecto sigue una arquitectura de tres capas estándar en Spring Boot. El frontend Angular (puerto 4200) se comunica exclusivamente con los Controllers a través de HTTP. Los Controllers delegan la lógica en los Services, y los Services usan los Repositories para persistir o consultar datos en PostgreSQL (puerto 5431).

Regla de oro: ningún Controller contiene lógica de negocio y ningún Repository sabe nada de HTTP. Todo el procesamiento inteligente vive en la capa Service.

2. El Motor de Riesgo — PerfilRiesgoService

Es el archivo más importante del proyecto. Cuando el Admin registra una nueva operación, este servicio ejecuta **una única consulta SQL** que cruza tres tablas de catálogo simultáneamente mediante LEFT JOINS.

Los tres vectores de riesgo:

Vector	Tabla cruzada	Condición de riesgo	Puntos
1 — Puerto Origen	catalogo_riesgo_pais	Nivel ALTO	+30 pts
		Nivel MEDIO	+15 pts
2 — Importador	importador_historial	Más de 2 infracciones previas	+40 pts
3 — Mercancía	restricciones_arancelarias	requiere_permiso = true	+30 pts

Canales de aforo resultantes:

Canal	Rango de puntaje	Acción requerida
VERDE	0 – 30 pts	Desaduanización automática. El estado pasa directamente a DESADUANIZACION.
AMARILLO	31 – 70 pts	Aforo documental. El Agente debe subir Factura, Certificado de Origen y Póliza de Seguro.
ROJO	71 – 100 pts	Aforo físico intrusivo. El Inspector debe abrir el contenedor y llenar el reporte de hallazgos.

Dato técnico: el puntaje total se calcula dentro de PostgreSQL usando expresiones CASE y COALESCE, no en código Java. Esto garantiza que si un parámetro no coincide con ningún catálogo, el LEFT JOIN devuelve NULL y COALESCE aplica 0 puntos automáticamente.

3. Resumen de Archivos Java

Archivo .java	Capa	Qué hace
PerfilRiesgoService	Service	Motor central. 1 query SQL con 3 LEFT JOINs calcula el score en PostgreSQL y asigna el canal VERDE/AMARILLO/ROJO.
EstadisticasService	Service	Calcula en tiempo real: producto más solicitado, puerto más riesgoso y empresa con más infracciones usando GROUP BY y COUNT.
OperacionAduanera Controller	Controller	Recibe peticiones HTTP para crear, listar, avanzar estado, ver análisis y eliminar operaciones.
UsuarioController	Controller	Gestiona login, registro y eliminación de personal (Agentes, Inspectores, Admins).
EstadisticasController	Controller	Expone GET /api/estadisticas — devuelve los datos calculados por EstadisticasService.
CatalogoController	Controller	Sirve los catálogos de puertos, importadores y arancelarios al formulario Angular.
DataInitializer	Config	Corre al arrancar el backend. Siembra los 13 puertos de Ecuador, España y USA, y los datos de ejemplo si la BD está vacía.
CorsConfig	Config	Permite que Angular (puerto 4200) llame al backend (puerto 8080) sin que el navegador bloquee las peticiones.
OperacionAduanera Usuario · Catalogo Importador · Restriccion	Model	Espejo de las tablas en PostgreSQL. JPA / Hibernate los sincroniza automáticamente con ddl-auto=update.
*Repository (5 archivos)	Repository	Interfaces JPA. Proveen findAll, findById, save, deleteById y existsById sin escribir SQL.

4. Módulo de Estadísticas — EstadisticasService

Calcula cuatro métricas en tiempo real mediante SQL con funciones de agregación. El endpoint GET /api/estadisticas es visible para todos los roles (Admin, Inspector, Agente) y el dashboard de Angular lo consulta al cargar.

Métrica	Técnica SQL
Producto más solicitado	GROUP BY codigo_arancelario + LEFT JOIN restricciones + ORDER BY COUNT DESC LIMIT 1
Puerto con más infracciones	GROUP BY puerto_origen + LEFT JOIN catalogo + SUM CASE canal AMARILLO/ROJO
Empresa más infractora	ORDER BY infracciones_previas DESC LIMIT 1 en importador_historial
Totales por canal	COUNT(*) + SUM CASE WHEN canal_aforo = 'VERDE'/'AMARILLO'/'ROJO'

5. Consultas SQL para Validar los Datos

Ejecuta estas queries en pgAdmin, DBeaver o cualquier cliente PostgreSQL conectado a tu base de datos. Están ordenadas de más simple a más completa.

Usuarios registrados

```
SELECT id, username, rol, cedula, nombre_completo FROM usuario ORDER BY id;
```

Catálogo de puertos cargado al arrancar el backend

```
SELECT id, nombre_puerto_o_pais, nivel_riesgo, puntos, motivo FROM catalogo_riesgo_pais  
ORDER BY nivel_riesgo, nombre_puerto_o_pais;
```

Importadores con sus infracciones

```
SELECT id, nombre_empresa, ruc_empresa, infracciones_previas, pais_origen FROM  
importador_historial ORDER BY infracciones_previas DESC;
```

Restricciones arancelarias (productos con/sin permiso especial)

```
SELECT codigo_arancelario, descripcion, categoria, requiere_permiso FROM  
restricciones_arancelarias ORDER BY requiere_permiso DESC, categoria;
```

Todas las operaciones con canal y puntaje

```
SELECT numero_tracking, tipo_operacion, puerto_origen, puerto_destino, canal_aforo,  
puntaje_riesgo, estado, fecha_registro FROM operaciones_aduaneras ORDER BY fecha_registro  
DESC;
```

Desglose completo del scoring — los mismos 3 JOINS que usa el motor

```
SELECT oa.numero_tracking, oa.puerto_origen, crp.nivel_riesgo, COALESCE(crp.puntos, 0) AS  
pts_origen, COALESCE(ih.infracciones_previas, 0) AS infracciones, CASE WHEN  
COALESCE(ih.infracciones_previas, 0) > 2 THEN 40 ELSE 0 END AS pts_importador,  
COALESCE(ra.requiere_permiso, false) AS permiso, CASE WHEN  
COALESCE(ra.requiere_permiso, false) THEN 30 ELSE 0 END AS pts_mercancia, oa.puntaje_riesgo,  
oa.canal_aforo FROM operaciones_aduaneras oa LEFT JOIN catalogo_riesgo_pais crp ON  
LOWER(TRIM(crp.nombre_puerto_o_pais)) = LOWER(TRIM(oa.puerto_origen)) LEFT JOIN  
importador_historial ih ON ih.id = oa.id_importador LEFT JOIN restricciones_arancelarias ra  
ON LOWER(TRIM(ra.codigo_arancelario)) = LOWER(TRIM(oa.codigo_arancelario)) ORDER BY  
oa.fecha_registro DESC;
```

Nota sobre la última query: esta es la más importante para validar. Te muestra exactamente los mismos cálculos que ejecuta PerfilRiesgoService cuando registra o analiza una operación. Si el canal mostrado en la app coincide con el de esta query, el motor funciona correctamente.